

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr. G. A. Senthil

QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

```
Command Prompt
C:\Users\ACER>py -3.12 --version
Python 3.12.10

C:\Users\ACER>py -3.12 -c "import sys; print(sys.executable)"
C:\Program Files\Python312\python.exe

C:\Users\ACER>mkdir RL-Lab

C:\Users\ACER>cd RL-Lab

C:\Users\ACER\RL-Lab>py -3.12 -m venv venv

C:\Users\ACER\RL-Lab>venv\Scripts\activate

(venv) C:\Users\ACER\RL-Lab>python -m pip install --upgrade pip
Requirement already satisfied: pip in c:\users\acer\rl-lab\venv\lib\site-packages (25.0.1)
Collecting pip
  Downloading pip-26.1.2-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.1.2-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 605.1 kB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.0.1
    Uninstalling pip-25.0.1:
      Successfully uninstalled pip-25.0.1
  Successfully installed pip-26.1.2

(venv) C:\Users\ACER\RL-Lab>pip install numpy
Collecting numpy
  Downloading numpy-2.5.1-cp312-cp312-win_amd64.whl.metadata (6.6 kB)
```

```
Command Prompt - python 3.12.10
=====
REINFORCEMENT LEARNING ENVIRONMENT VERIFICATION
=====

1. Python Version
3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)]

2. NumPy Version
2.5.1

3. Matplotlib Version
3.11.0

4. Gymnasium Version
1.3.0

5. TensorFlow Version
2.21.0

6. Keras Version
3.15.0

Creating CartPole Environment...

Initial State:
[ 0.01226248  0.03081428 -0.04663165 -0.0194387 ]

Action Space: Discrete(2)
Observation Space: Box([-4.8          -inf -0.41887903   -inf], [4.8          inf  0.41887903    inf], (4,), float32)

Random Action Selected: 0

Next State:
[ 0.01287877 -0.16360897 -0.04702042  0.25817442]

Reward: 1.0
Terminated: False
Truncated: False

Testing Matplotlib...

experiment1.PY
File Edit View

# Experiment 1
# Aim: Installation and Verification of Reinforcement Learning Development Environment

import sys
import numpy as np
import matplotlib.pyplot as plt
import gymnasium as gym
import tensorflow as tf
import keras

print("\n*****")
print("REINFORCEMENT LEARNING ENVIRONMENT VERIFICATION")
print("\n*****")

print("\n1. Python Version")
print(sys.version)

print("\n2. NumPy Version")
print(np.__version__)

print("\n3. Matplotlib Version")
print(plt.matplotlib.__version__)

print("\n4. Gymnasium Version")
print(gym.__version__)

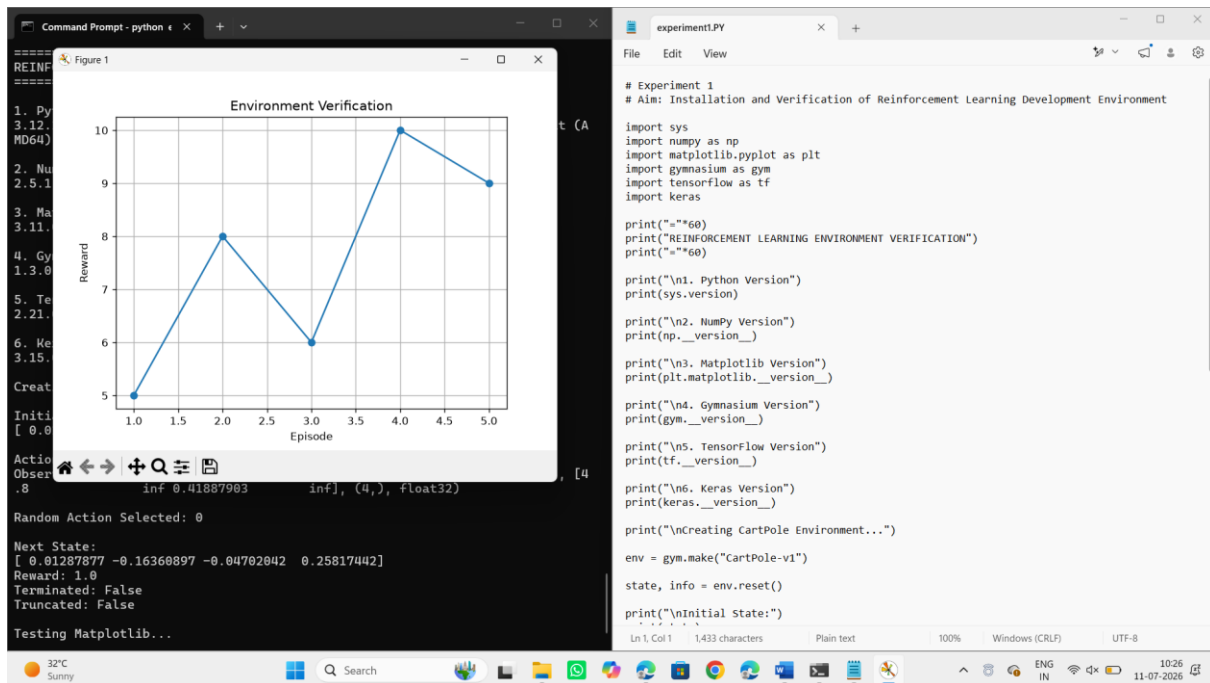
print("\n5. TensorFlow Version")
print(tf.__version__)

print("\n6. Keras Version")
print(keras.__version__)

print("\nCreating CartPole Environment...")

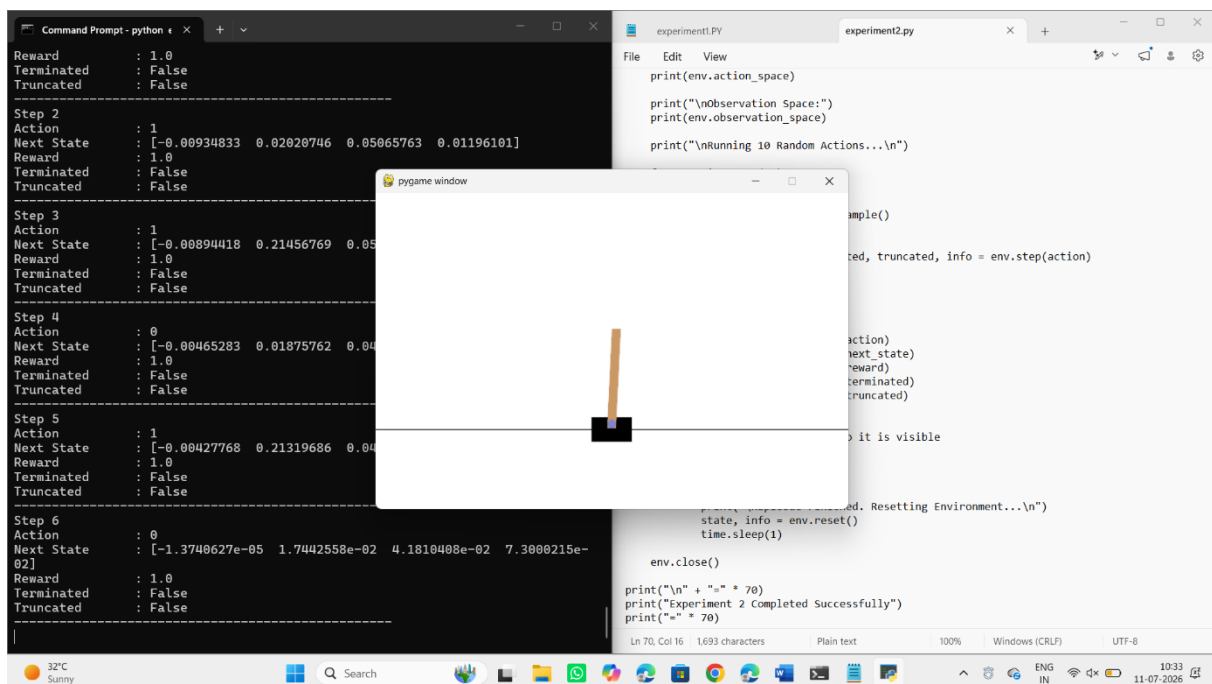
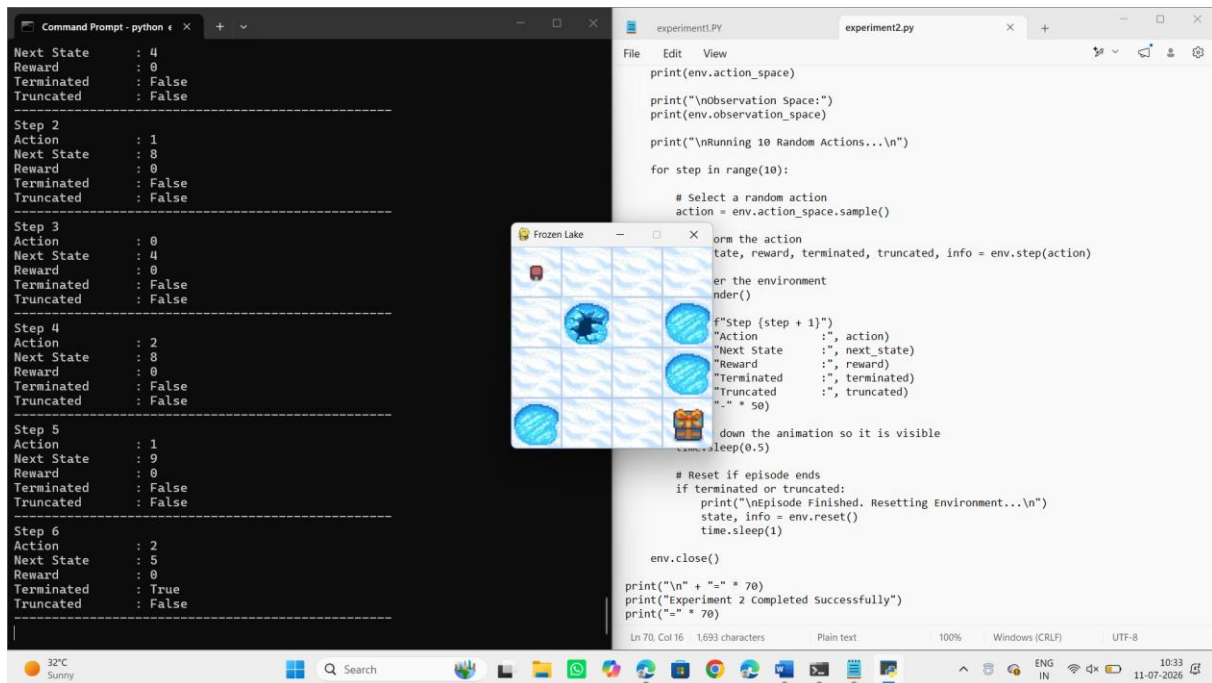
env = gym.make("CartPole-v1")
state, info = env.reset()

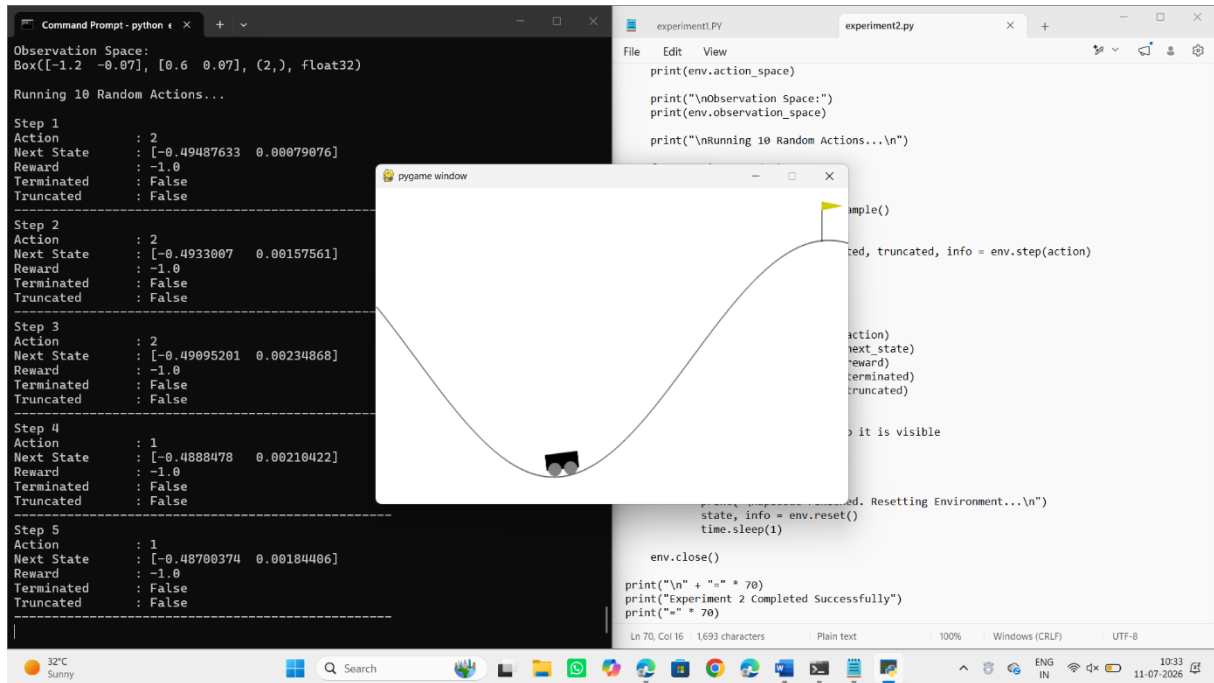
print("\nInitial State:")
print(state)
```



Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

Aim: To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium to understand the concepts of states, actions, rewards, and episodes.





```

Command Prompt - python
Observation Space:
Box([-1.2 -0.07], [0.6 0.07], (2,), float32)

Running 10 Random Actions...

Step 1
Action      : 2
Next State  : [-0.49487633  0.00879076]
Reward     : -1.0
Terminated  : False
Truncated  : False

Step 2
Action      : 2
Next State  : [-0.4933007  0.00157561]
Reward     : -1.0
Terminated  : False
Truncated  : False

Step 3
Action      : 2
Next State  : [-0.49095201  0.00234868]
Reward     : -1.0
Terminated  : False
Truncated  : False

Step 4
Action      : 1
Next State  : [-0.4888478  0.00210422]
Reward     : -1.0
Terminated  : False
Truncated  : False

Step 5
Action      : 1
Next State  : [-0.48700374  0.00184406]
Reward     : -1.0
Terminated  : False
Truncated  : False

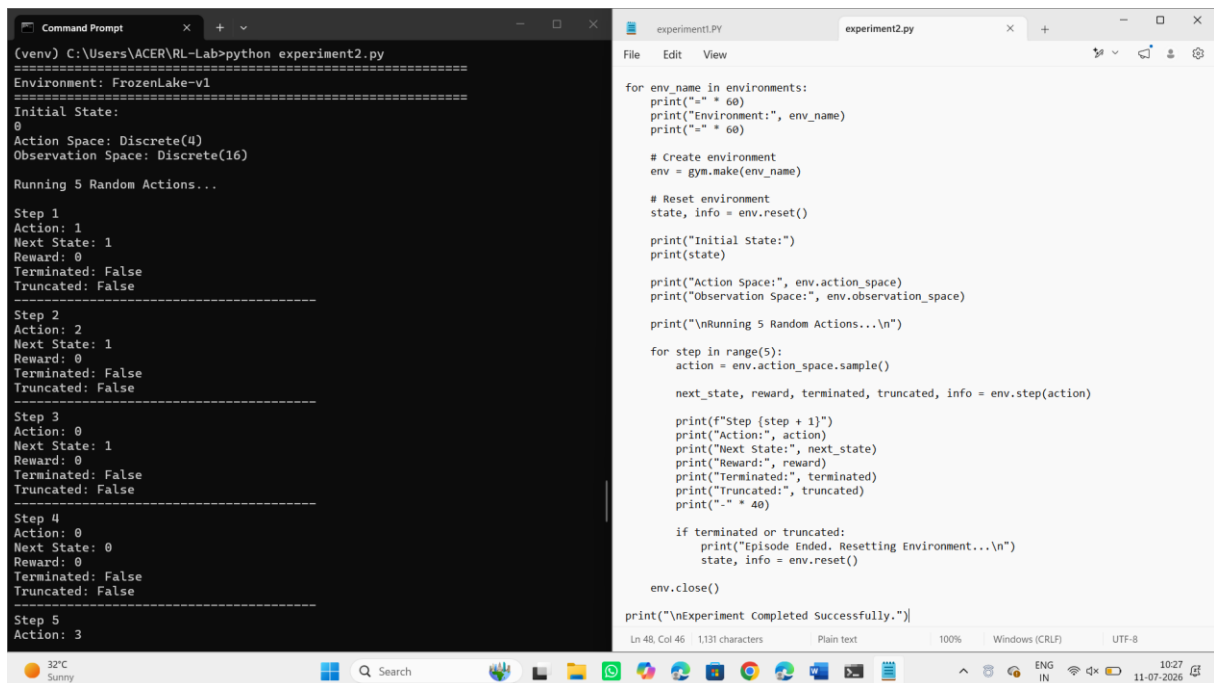
pygame window
A 2D plot showing a sine wave. A small car-like object is positioned at the bottom center of the plot.

experiment1.py
File Edit View
print(env.action_space)
print("\nObservation Space:")
print(env.observation_space)
print("\nRunning 10 Random Actions...\n")

...

env.close()
print("\n" + "-" * 70)
print("Experiment 2 Completed Successfully")
print("\n" + "-" * 70)
Ln 70, Col 16 1,693 characters Plain text 100% Windows (CRLF) UTF-8
1033 11-07-2026

```



```

Command Prompt
(venv) C:\Users\ACER\RL-Lab>python experiment2.py
=====
Environment: FrozenLake-v1
=====
Initial State:
0
Action Space: Discrete(4)
Observation Space: Discrete(16)

Running 5 Random Actions...

Step 1
Action: 1
Next State: 1
Reward: 0
Terminated: False
Truncated: False

Step 2
Action: 2
Next State: 1
Reward: 0
Terminated: False
Truncated: False

Step 3
Action: 0
Next State: 1
Reward: 0
Terminated: False
Truncated: False

Step 4
Action: 0
Next State: 0
Reward: 0
Terminated: False
Truncated: False

Step 5
Action: 3
=====

experiment2.py
File Edit View
for env_name in environments:
    print("-" * 60)
    print("Environment:", env_name)
    print("-" * 60)

    # Create environment
    env = gym.make(env_name)

    # Reset environment
    state, info = env.reset()

    print("Initial State:")
    print(state)

    print("Action Space:", env.action_space)
    print("Observation Space:", env.observation_space)

    print("\nRunning 5 Random Actions...\n")

    for step in range(5):
        action = env.action_space.sample()

        next_state, reward, terminated, truncated, info = env.step(action)

        print(f"Step {step + 1}")
        print("Action:", action)
        print("Next State:", next_state)
        print("Reward:", reward)
        print("Terminated:", terminated)
        print("Truncated:", truncated)
        print("-" * 40)

        if terminated or truncated:
            print("Episode Ended. Resetting Environment...\n")
            state, info = env.reset()

    env.close()

print("\nExperiment Completed Successfully.")
Ln 48, Col 46 1,131 characters Plain text 100% Windows (CRLF) UTF-8
1027 11-07-2026

```

```

=====
Environment: CartPole-v1
=====
Initial State:
[-0.02775776  0.04603251  0.02235265 -0.03090249]
Action Space: Discrete(2)
Observation Space: Box([-4.8          -inf -0.41887903      -inf], [4.8          inf  0.41887903      inf], (4,), float32)

Running 5 Random Actions...

Step 1
Action: 0
Next State: [-0.02683711 -0.14940274  0.0217346  0.26874828]
Reward: 1.0
Terminated: False
Truncated: False

Step 2
Action: 1
Next State: [-0.02982517  0.04540241  0.02710957 -0.01700095]
Reward: 1.0
Terminated: False
Truncated: False

Step 3
Action: 0
Next State: [-0.02891712 -0.15009762  0.02676955  0.28411058]
Reward: 1.0
Terminated: False
Truncated: False

Step 4
Action: 0
Next State: [-0.03191907 -0.34559095  0.03245176  0.58511484]
Reward: 1.0
Terminated: False
Truncated: False

Step 5

```

```

experiment1.PY
experiment2.py

import gymnasium as gym

# List of environments
environments = [
    "FrozenLake-v1",
    "CartPole-v1",
    "MountainCar-v0"
]

for env_name in environments:
    print("=" * 60)
    print("Environment:", env_name)
    print("=" * 60)

    # Create environment
    env = gym.make(env_name)

    # Reset environment
    state, info = env.reset()

    print("Initial State:")
    print(state)

    print("Action Space:", env.action_space)
    print("Observation Space:", env.observation_space)

    print("\nRunning 5 Random Actions...\n")

    for step in range(5):
        action = env.action_space.sample()

        next_state, reward, terminated, truncated, info = env.step(action)

        print(f"Step (step + 1)")
        print("Action:", action)
        print("Next State:", next_state)
        print("Reward:", reward)
        print("Terminated:", terminated)
        print("Truncated:", truncated)

```

32°C Sunny

Search

11-07-2026 10:28

```

=====
Environment: MountainCar-v0
=====
Initial State:
[-0.5399643  0.          ]
Action Space: Discrete(3)
Observation Space: Box([-1.2 -0.07], [0.6  0.07], (2,), float32)

Running 5 Random Actions...

Step 1
Action: 1
Next State: [-5.3984165e-01  1.2269235e-04]
Reward: -1.0
Terminated: False
Truncated: False

Step 2
Action: 2
Next State: [-0.53859717  0.00124447]
Reward: -1.0
Terminated: False
Truncated: False

Step 3
Action: 0
Next State: [-5.3824025e-01  3.5691564e-04]
Reward: -1.0
Terminated: False
Truncated: False

Step 4
Action: 1
Next State: [-5.3777355e-01  4.6669145e-04]
Reward: -1.0
Terminated: False
Truncated: False

Step 5
Action: 1
Next State: [-0.53720057  0.00057297]

```

```

experiment1.PY
experiment2.py

import gymnasium as gym

# List of environments
environments = [
    "FrozenLake-v1",
    "CartPole-v1",
    "MountainCar-v0"
]

for env_name in environments:
    print("=" * 60)
    print("Environment:", env_name)
    print("=" * 60)

    # Create environment
    env = gym.make(env_name)

    # Reset environment
    state, info = env.reset()

    print("Initial State:")
    print(state)

    print("Action Space:", env.action_space)
    print("Observation Space:", env.observation_space)

    print("\nRunning 5 Random Actions...\n")

    for step in range(5):
        action = env.action_space.sample()

        next_state, reward, terminated, truncated, info = env.step(action)

        print(f"Step (step + 1)")
        print("Action:", action)
        print("Next State:", next_state)
        print("Reward:", reward)
        print("Terminated:", terminated)
        print("Truncated:", truncated)

```

32°C Sunny

Search

11-07-2026 10:28

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

```

Command Prompt - python
Current State : [-0.00564598 -0.95190954 0.02453546 1.3293623 ]
Action       : 1
Next State   : [-0.02468418 -0.75718565 0.0511227  1.044457 ]
Reward       : 1.0

Step 13
Current State : [-0.02468418 -0.75718565 0.0511227  1.044457 ]
Action       : 1
Next State   : [-0.03982629 -0.5626983  0.07203508  1.087338 ]
Reward       : 1.0

Step 14
Current State : [-0.03982629 -0.5626983  0.07203508  1.087338 ]
Action       : 0
Next State   : [-0.05108026 -0.7587338  0.08733808  1.087338 ]
Reward       : 1.0

Step 15
Current State : [-0.05108026 -0.7587338  0.08733808  1.087338 ]
Action       : 1
Next State   : [-0.06625493 -0.5648668  0.10903075  0.81865984 ]
Reward       : 1.0

Step 16
Current State : [-0.06625493 -0.5648668  0.10903075  0.81865984 ]
Action       : 0
Next State   : [-0.07755227 -0.7612987  0.12540394  1.1435497 ]
Reward       : 1.0

Step 17
Current State : [-0.07755227 -0.7612987  0.12540394  1.1435497 ]
Action       : 0
Next State   : [-0.09277824 -0.9578158  0.14827494  1.4727827 ]
Reward       : 1.0

Step 18
Current State : [-0.09277824 -0.9578158  0.14827494  1.4727827 ]
Action       : 1
Next State   : [-0.11193456 -0.76478493  0.17773059  1.2298467 ]
Reward       : 1.0

pygame window
[Environment visualization showing a stick figure]

experiment3.py
total_reward = 0
step = 0

while True:
    # Random policy (agent chooses a random action)
    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    # Display environment
    env.render()
    time.sleep(0.2)

    total_reward += reward
    step += 1

    print(f"\nStep (step)")
    print("Current State :", state)
    print("Action       :", action)
    print("Next State   :", next_state)
    print("Reward       :", reward)

    # Update current state
    state = next_state

    # Check if episode has ended
    if terminated or truncated:
        print("\nEpisode Finished")
        print("Total Steps :", step)
        print("Total Reward :", total_reward)
        break

env.close()

print("\n" + "=" * 70)
print("Experiment 3 Completed Successfully")
print("=" * 70)

```

```

Command Prompt
Reward       : 1.0

Step 16
Current State : [-0.06625493 -0.5648668  0.10903075  0.81865984 ]
Action       : 0
Next State   : [-0.07755227 -0.7612987  0.12540394  1.1435497 ]
Reward       : 1.0

Step 17
Current State : [-0.07755227 -0.7612987  0.12540394  1.1435497 ]
Action       : 0
Next State   : [-0.09277824 -0.9578158  0.14827494  1.4727827 ]
Reward       : 1.0

Step 18
Current State : [-0.09277824 -0.9578158  0.14827494  1.4727827 ]
Action       : 1
Next State   : [-0.11193456 -0.76478493  0.17773059  1.2298467 ]
Reward       : 1.0

Step 19
Current State : [-0.11193456 -0.76478493  0.17773059  1.2298467 ]
Action       : 0
Next State   : [-0.12723026 -0.96169126  0.20232753  1.5725317 ]
Reward       : 1.0

Step 20
Current State : [-0.12723026 -0.96169126  0.20232753  1.5725317 ]
Action       : 1
Next State   : [-0.14646408 -0.7694756  0.23377816  1.3491688 ]
Reward       : 1.0

Episode Finished
Total Steps : 20
Total Reward : 20.0

=====
Experiment 3 Completed Successfully
=====

(venv) C:\Users\ACER\RL-Lab>

experiment3.py
total_reward = 0
step = 0

while True:
    # Random policy (agent chooses a random action)
    action = env.action_space.sample()

    # Perform action
    next_state, reward, terminated, truncated, info = env.step(action)

    # Display environment
    env.render()
    time.sleep(0.2)

    total_reward += reward
    step += 1

    print(f"\nStep (step)")
    print("Current State :", state)
    print("Action       :", action)
    print("Next State   :", next_state)
    print("Reward       :", reward)

    # Update current state
    state = next_state

    # Check if episode has ended
    if terminated or truncated:
        print("\nEpisode Finished")
        print("Total Steps :", step)
        print("Total Reward :", total_reward)
        break

env.close()

print("\n" + "=" * 70)
print("Experiment 3 Completed Successfully")
print("=" * 70)

```

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

```
Command Prompt
Next State : [-0.12723026 -0.96169126 0.20232753 1.5725317 ]
Reward : 1.0

Step 20
Current State : [-0.12723026 -0.96169126 0.20232753 1.5725317 ]
Action : 1
Next State : [-0.14646408 -0.7694756 0.23377816 1.3491688 ]
Reward : 1.0

Episode Finished
Total Steps : 20
Total Reward : 20.0

=====
Experiment 3 Completed Successfully
=====

(venv) C:\Users\ACER\VL-Lab>python experiment6.py
=====
EXPLORATION vs EXPLOITATION
=====

Random Strategy
Total Reward : 126

Greedy Strategy
Best Action : 3
Total Reward : 200

Epsilon-Greedy Strategy
Estimated Q-values : [4.999990463256836, 0, 1.5, 0, 0]
Total Reward : 98

=====
Comparison
=====
Random Strategy Total Reward : 126
Greedy Strategy Total Reward : 200
Epsilon-Greedy Total Reward : 98

(venv) C:\Users\ACER\VL-Lab>
```

```
# Experiment 6
# Aim: Exploration vs Exploitation Strategies

import random

# Reward values for 5 actions (unknown to the agent)
true_rewards = [5, 8, 3, 10, 6]

# Estimated Q-values
q_values = [0] * 5

epsilon = 0.2
episodes = 20

print("=" * 60)
print("EXPLORATION vs EXPLOITATION")
print("=" * 60)

random_total = 0
greedy_total = 0
epsilon_total = 0

# ----- Random Strategy -----
print("\nRandom Strategy")

for _ in range(episodes):
    action = random.randint(0, 4)
    reward = true_rewards[action]
    random_total += reward

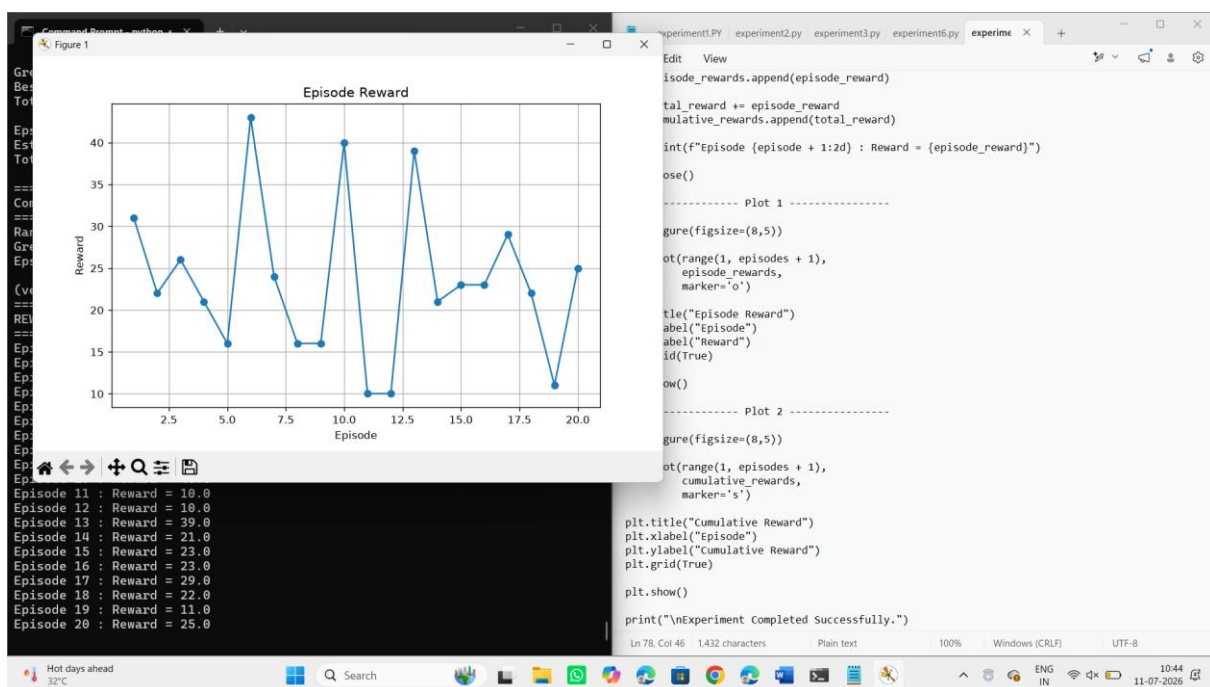
print("Total Reward :", random_total)

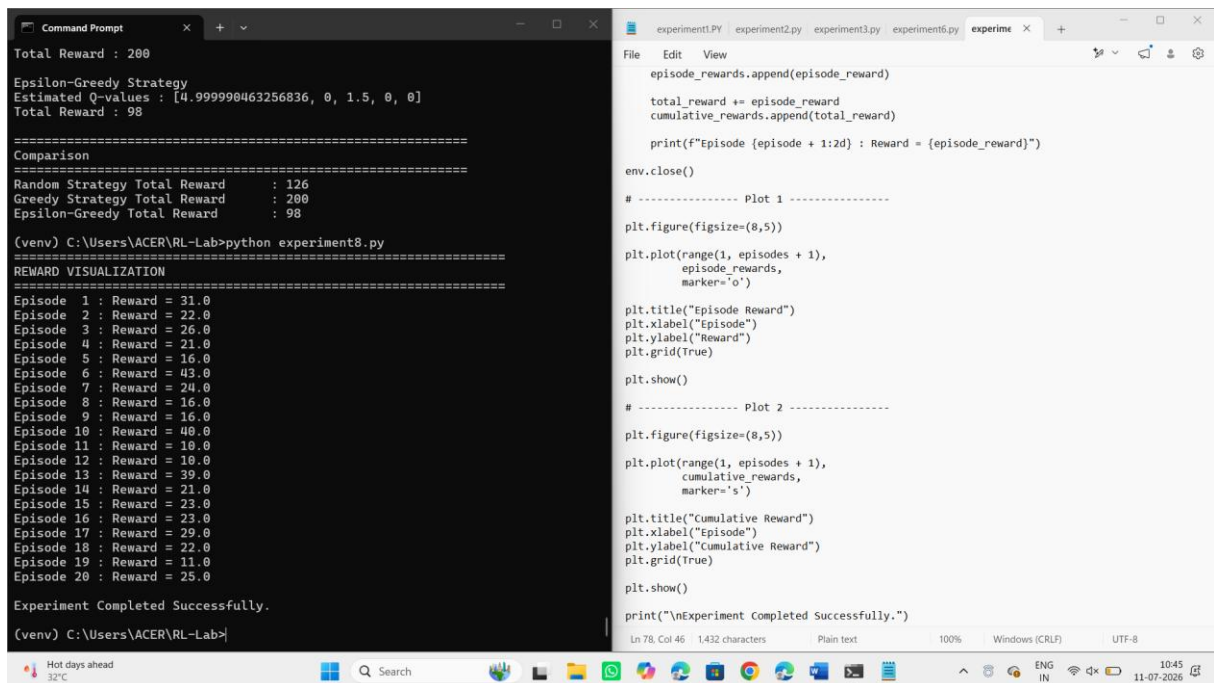
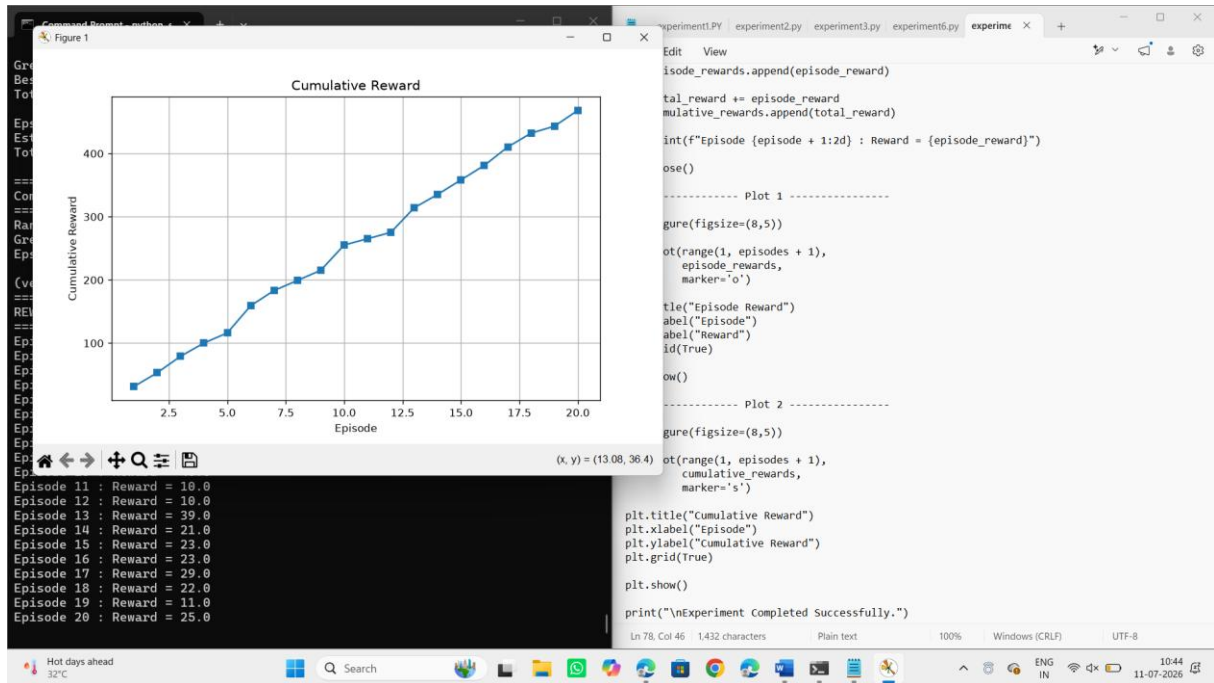
# ----- Greedy Strategy -----
print("\nGreedy Strategy")

best_action = true_rewards.index(max(true_rewards))
```

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.





Code of Conduct:

I _S.Sirivally / 192472371_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Sirivally.S

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

